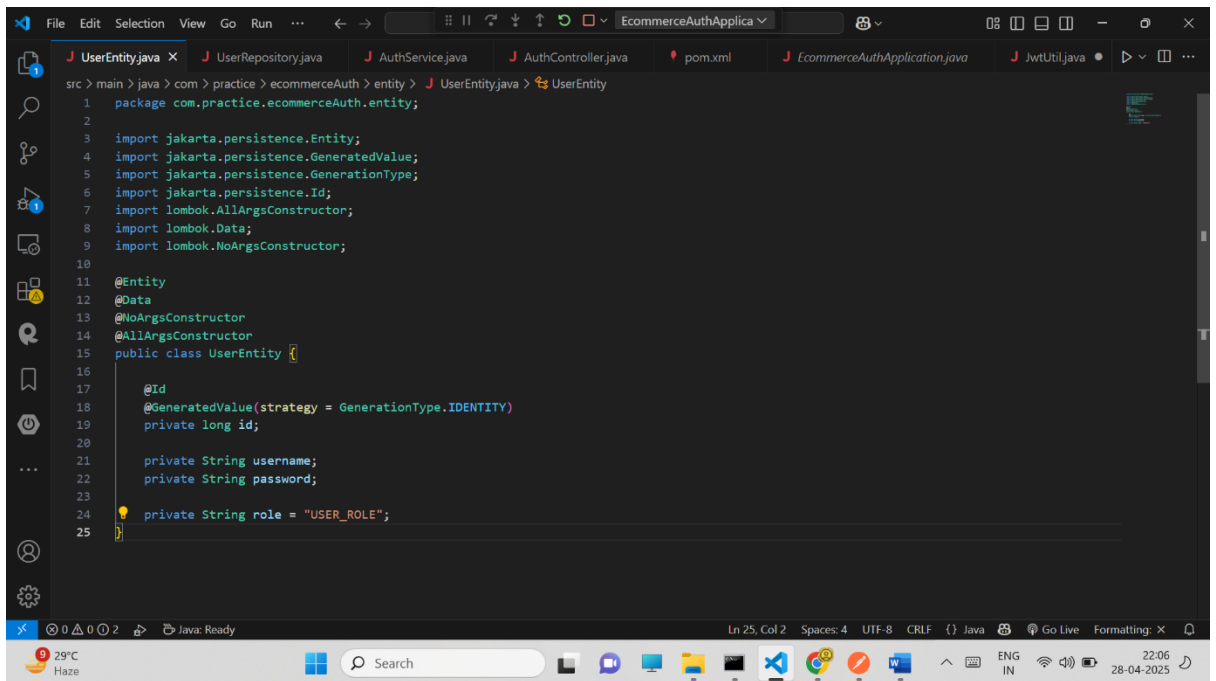
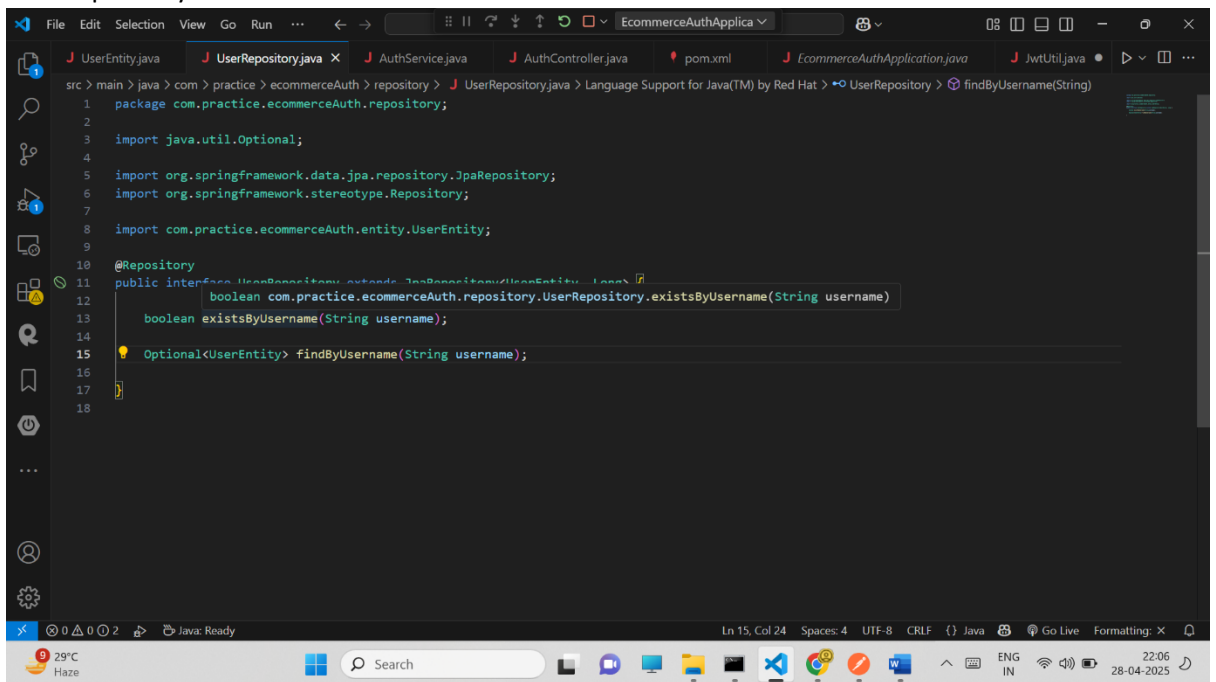


1. UserEntity



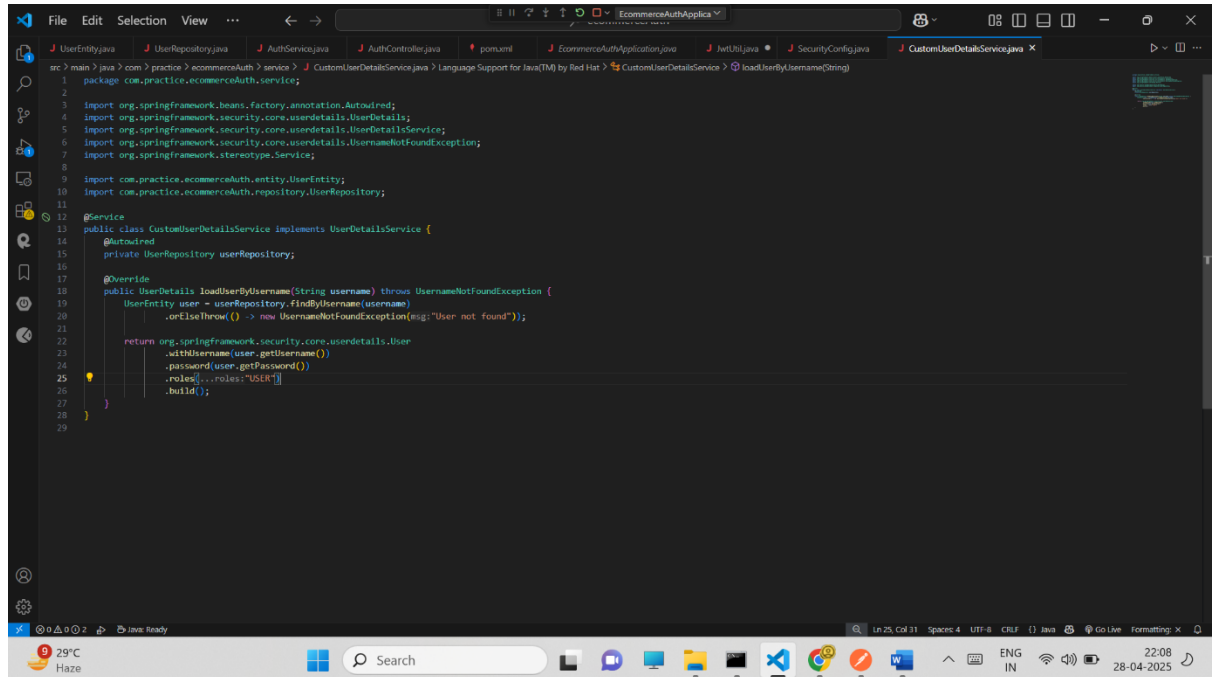
```
src > main > java > com > practice > ecommerceAuth > entity > J UserEntity.java > UserEntity
1 package com.practice.ecommerceAuth.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import lombok.AllArgsConstructor;
8 import lombok.Data;
9 import lombok.NoArgsConstructor;
10
11 @Entity
12 @Data
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public class UserEntity {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private long id;
20
21     private String username;
22     private String password;
23
24     private String role = "USER_ROLE";
25 }
```

2. UserRepository



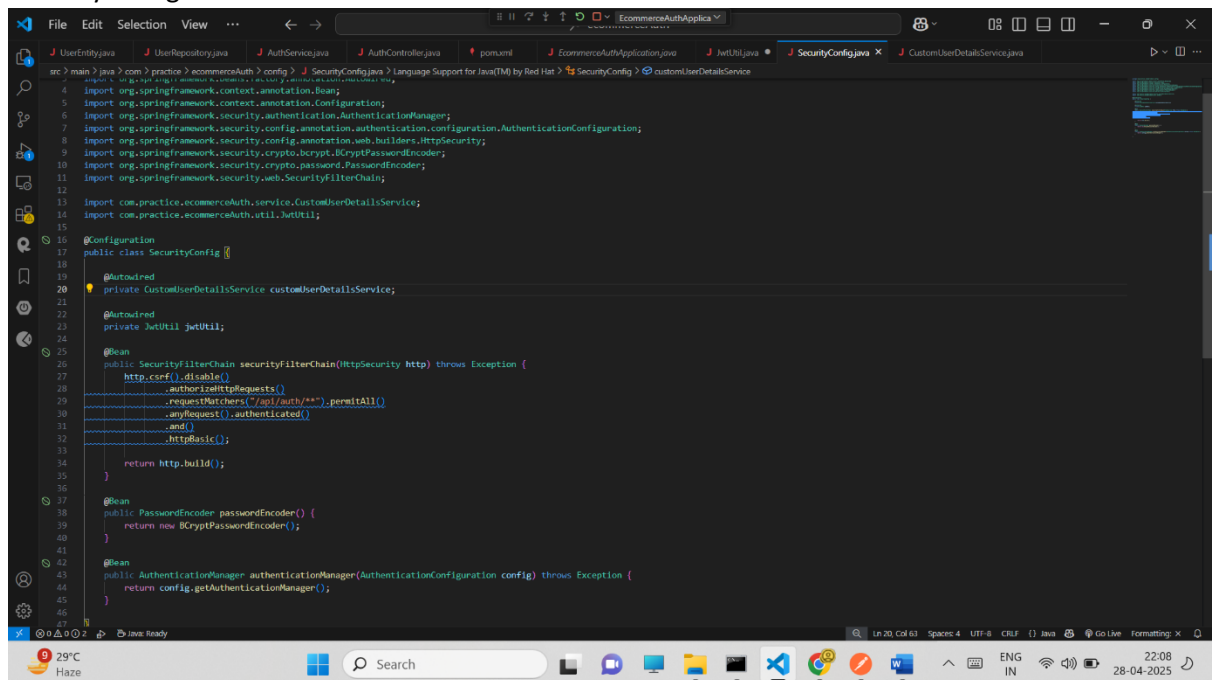
```
src > main > java > com > practice > ecommerceAuth > repository > J UserRepository.java > Language Support for Java(TM) by Red Hat > UserRepository > findByUsername(String)
1 package com.practice.ecommerceAuth.repository;
2
3 import java.util.Optional;
4
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 import com.practice.ecommerceAuth.entity.UserEntity;
9
10 @Repository
11 public interface UserRepository extends JpaRepository<UserEntity, Long> {
12     boolean existsByUsername(String username);
13     boolean existsByUsername(String username);
14
15     Optional<UserEntity> findByUsername(String username);
16
17 }
18 }
```

3. CustomUserDetailsService



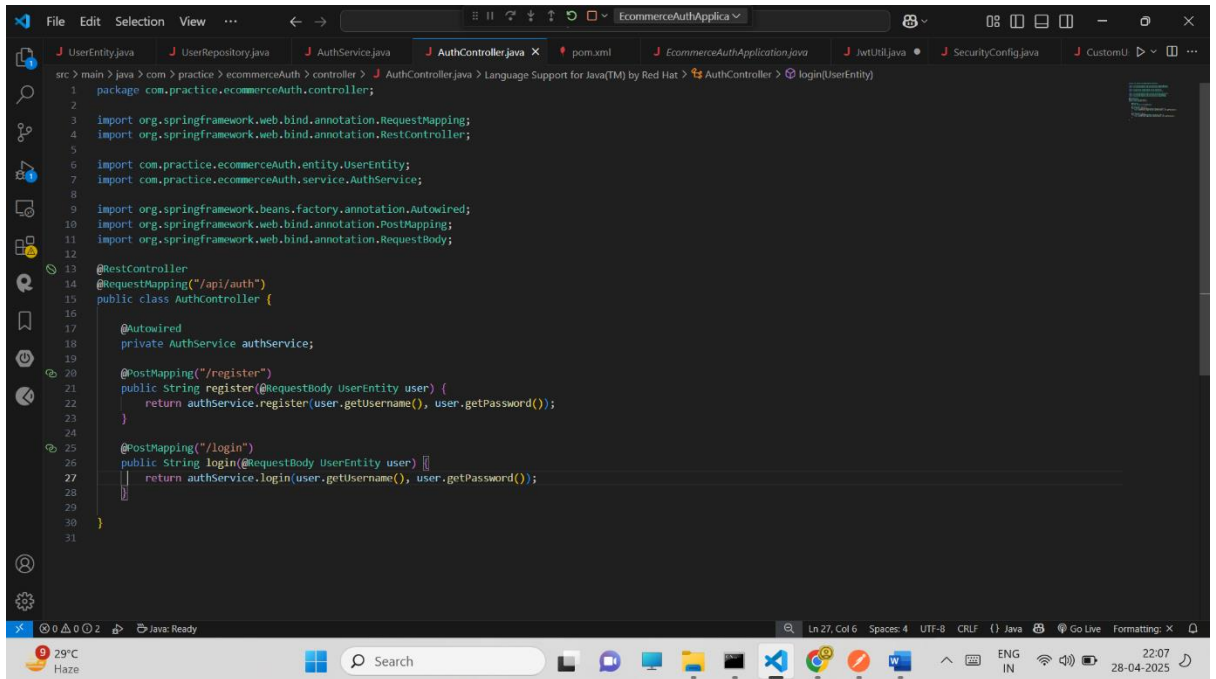
```
src > main > java > com > practice > ecommerceAuth > service > CustomUserDetailsService.java > Language Support for Java(TM) by Red Hat > CustomUserDetailsService > loadUserByUsername(String)
1 package com.practice.ecommerceAuth.service;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.security.core.userdetails.UserDetails;
5 import org.springframework.security.core.userdetails.UserDetailsService;
6 import org.springframework.security.core.userdetails.UsernameNotFoundException;
7 import org.springframework.stereotype.Service;
8
9 import com.practice.ecommerceAuth.entity.UserEntity;
10 import com.practice.ecommerceAuth.repository.UserRepository;
11
12 @Service
13 public class CustomUserDetailsService implements UserDetailsService {
14     @Autowired
15     private UserRepository userRepository;
16
17     @Override
18     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
19         UserEntity user = userRepository.findByUsername(username)
20             .orElseThrow(() -> new UsernameNotFoundException(msg:"User not found"));
21
22         return org.springframework.security.core.userdetails.User
23             .withUsername(user.getUsername())
24             .password(user.getPassword())
25             .roles(...roles:"USER")
26             .build();
27     }
28
29 }
```

4. SecurityConfig



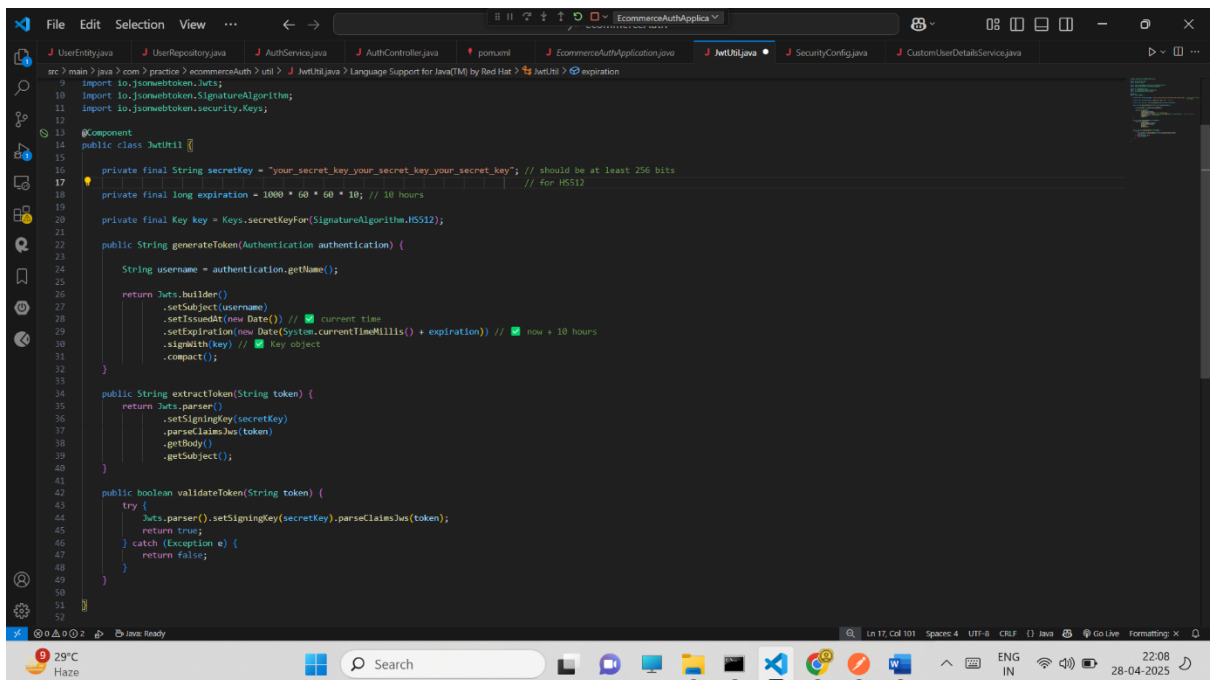
```
src > main > java > com > practice > ecommerceAuth > config > SecurityConfig.java > Language Support for Java(TM) by Red Hat > SecurityConfig > customUserDetailsService
4 import org.springframework.context.annotation.Bean;
5 import org.springframework.context.annotation.Configuration;
6 import org.springframework.security.authentication.AuthenticationManager;
7 import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
8 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
9 import org.springframework.security.config.annotation.web.configuration.annotation.builders.HttpSecurity;
10 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
11 import org.springframework.security.crypto.password.PasswordEncoder;
12 import org.springframework.security.web.SecurityFilterChain;
13
14 import com.practice.ecommerceAuth.service.CustomUserDetailsService;
15 import com.practice.ecommerceAuth.util.JwtUtil;
16
17 @Configuration
18 public class SecurityConfig {
19     @Autowired
20     private CustomUserDetailsService customUserDetailsService;
21
22     @Autowired
23     private JwtUtil jwtUtil;
24
25     @Bean
26     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
27         http.csrf().disable()
28             .authorizeHttpRequests()
29             .requestMatchers("/api/auth/**").permitAll()
30             .anyRequest().authenticated()
31             .and()
32             .httpBasic();
33         return http.build();
34     }
35
36     @Bean
37     public PasswordEncoder passwordEncoder() {
38         return new BCryptPasswordEncoder();
39     }
40
41     @Bean
42     public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
43         return config.getAuthenticationManager();
44     }
45
46 }
```

5. AuthController



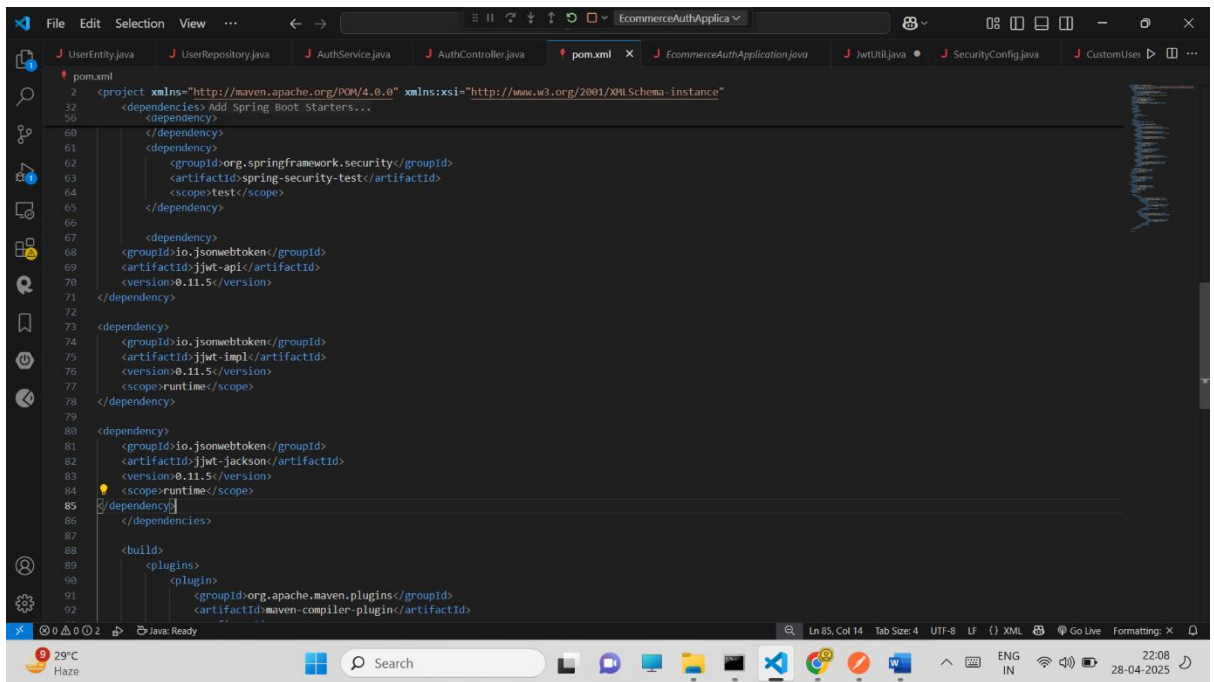
```
src > main > java > com > practice > ecommerceAuth > controller > J AuthController.java > Language Support for Java(TM) by Red Hat > J AuthController > login(UserEntity)
1 package com.practice.ecommerceAuth.controller;
2
3 import org.springframework.web.bind.annotation.RequestMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 import com.practice.ecommerceAuth.entity.UserEntity;
7 import com.practice.ecommerceAuth.service.AuthService;
8
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.web.bind.annotation.PostMapping;
11 import org.springframework.web.bind.annotation.RequestBody;
12
13 @RestController
14 @RequestMapping("/api/auth")
15 public class AuthController {
16
17     @Autowired
18     private AuthService authService;
19
20     @PostMapping("/register")
21     public String register(@RequestBody UserEntity user) {
22         return authService.register(user.getUsername(), user.getPassword());
23     }
24
25     @PostMapping("/login")
26     public String login(@RequestBody UserEntity user) {
27         return authService.login(user.getUsername(), user.getPassword());
28     }
29
30 }
31
```

6. JwtUtil



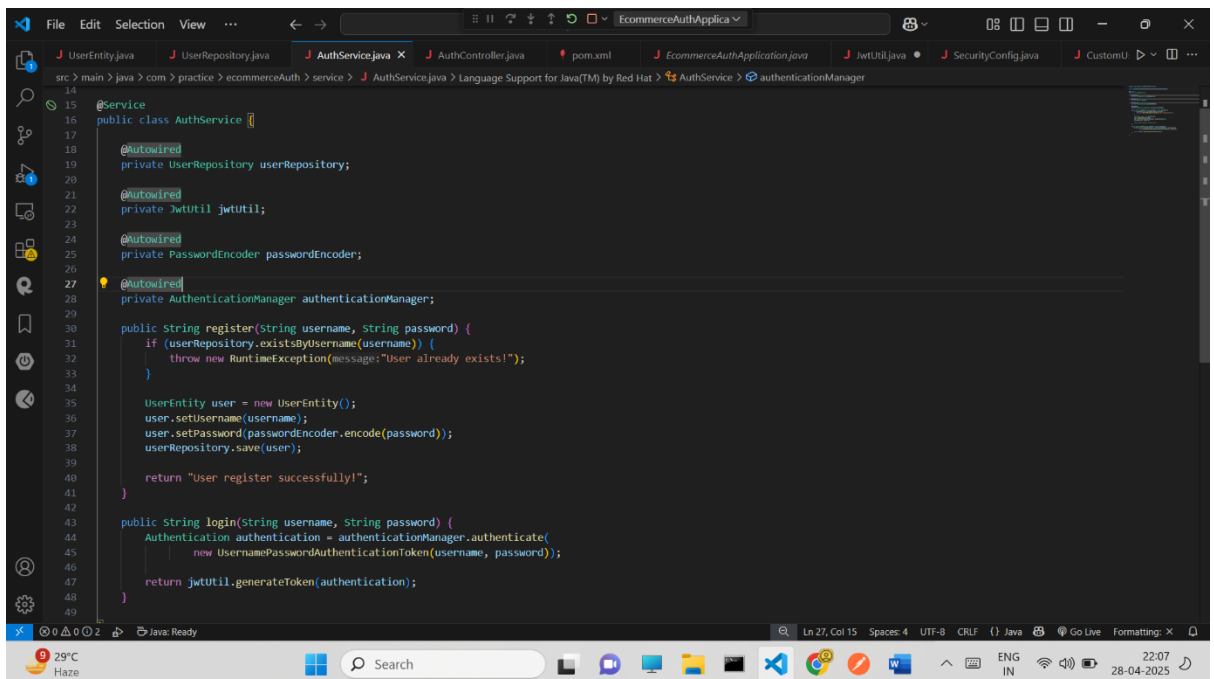
```
src > main > java > com > practice > ecommerceAuth > util > J JwtUtil.java > Language Support for Java(TM) by Red Hat > J JwtUtil > expiration
9 import io.jsonwebtoken.Jwts;
10 import io.jsonwebtoken.SignatureAlgorithm;
11 import io.jsonwebtoken.security.Keys;
12
13 @Component
14 public class JwtUtil {
15
16     private final String secretKey = "your_secret_key_your_secret_key_your_secret_key"; // should be at least 256 bits
17     private final long expiration = 1000 * 60 * 60 * 10; // 10 hours // for HS512
18
19     private final Key key = Keys.secretKeyFor(SignatureAlgorithm.HS512);
20
21     public String generateToken(Authentication authentication) {
22
23         String username = authentication.getName();
24
25         return Jwts.builder()
26             .setSubject(username)
27             .setIssuedAt(new Date()) // [X] current time
28             .setExpiration(new Date(System.currentTimeMillis() + expiration)) // [X] now + 10 hours
29             .signWith(key) // [X] key object
30             .compact();
31     }
32
33     public String extractToken(String token) {
34         return Jwts.parser()
35             .setSigningKey(secretKey)
36             .parseClaimsJws(token)
37             .getBody()
38             .getSubject();
39     }
40
41     public boolean validateToken(String token) {
42         try {
43             Jwts.parser().setSigningKey(secretKey).parseClaimsJws(token);
44             return true;
45         } catch (Exception e) {
46             return false;
47         }
48     }
49
50 }
51
52
```

7. Pom.xml



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <groupId>com.practice</groupId>
6     <artifactId>EcommerceAuthApplica</artifactId>
7     <version>0.0.1-SNAPSHOT</version>
8     <packaging>war</packaging>
9     <name>EcommerceAuthApplica</name>
10    <description>EcommerceAuthApplica</description>
11    <parent>
12        <groupId>com.practice</groupId>
13        <artifactId>EcommerceAuthApplica</artifactId>
14        <version>0.0.1-SNAPSHOT</version>
15    </parent>
16    <dependencies>
17        <dependency>
18            <groupId>org.springframework.boot</groupId>
19            <artifactId>spring-boot-starter</artifactId>
20        </dependency>
21        <dependency>
22            <groupId>org.springframework.boot</groupId>
23            <artifactId>spring-boot-starter-web</artifactId>
24        </dependency>
25        <dependency>
26            <groupId>org.springframework.boot</groupId>
27            <artifactId>spring-boot-starter-security</artifactId>
28        </dependency>
29        <dependency>
30            <groupId>io.jsonwebtoken</groupId>
31            <artifactId>jjwt-api</artifactId>
32            <version>0.11.5</version>
33        </dependency>
34        <dependency>
35            <groupId>io.jsonwebtoken</groupId>
36            <artifactId>jjwt-impl</artifactId>
37            <version>0.11.5</version>
38            <scope>runtime</scope>
39        </dependency>
40        <dependency>
41            <groupId>io.jsonwebtoken</groupId>
42            <artifactId>jjwt-jackson</artifactId>
43            <version>0.11.5</version>
44            <scope>runtime</scope>
45        </dependency>
46    </dependencies>
47    <build>
48        <plugins>
49            <plugin>
50                <groupId>org.apache.maven.plugins</groupId>
51                <artifactId>maven-compiler-plugin</artifactId>
52                <version>3.10.0</version>
53                <configuration>
54                    <source>11</source>
55                    <target>11</target>
56                </configuration>
57            </plugin>
58        </plugins>
59    </build>
60 </project>
```

8. AuthService



```
14 src > main > java > com > practice > ecommerceAuth > service > J AuthService.java > Language Support for Java(TM) by Red Hat > AuthService > authenticationManager
15 @Service
16 public class AuthService {
17
18     @Autowired
19     private UserRepository userRepository;
20
21     @Autowired
22     private JwtUtil jwtUtil;
23
24     @Autowired
25     private PasswordEncoder passwordEncoder;
26
27     @Autowired
28     private AuthenticationManager authenticationManager;
29
30     public String register(String username, String password) {
31         if (userRepository.existsByUsername(username)) {
32             throw new RuntimeException(message:"User already exists!");
33         }
34
35         UserEntity user = new UserEntity();
36         user.setUsername(username);
37         user.setPassword(passwordEncoder.encode(password));
38         userRepository.save(user);
39
40         return "User register successfully!";
41     }
42
43     public String login(String username, String password) {
44         Authentication authentication = authenticationManager.authenticate(
45             new UsernamePasswordAuthenticationToken(username, password));
46
47         return jwtUtil.generateToken(authentication);
48     }
49 }
```